SHOP'IMT serverless : LLD

Oct 27, 2026 · @Louison

La solution serverless de SHOP'IMT est déployée et testée en eu-west-3. Une commande passe en environ 30 secondes de « reçue » à « livrée », avec un incident de transport tiré au sort. Le coût reste à 0 € dans le Free Tier.

Objet et périmètre

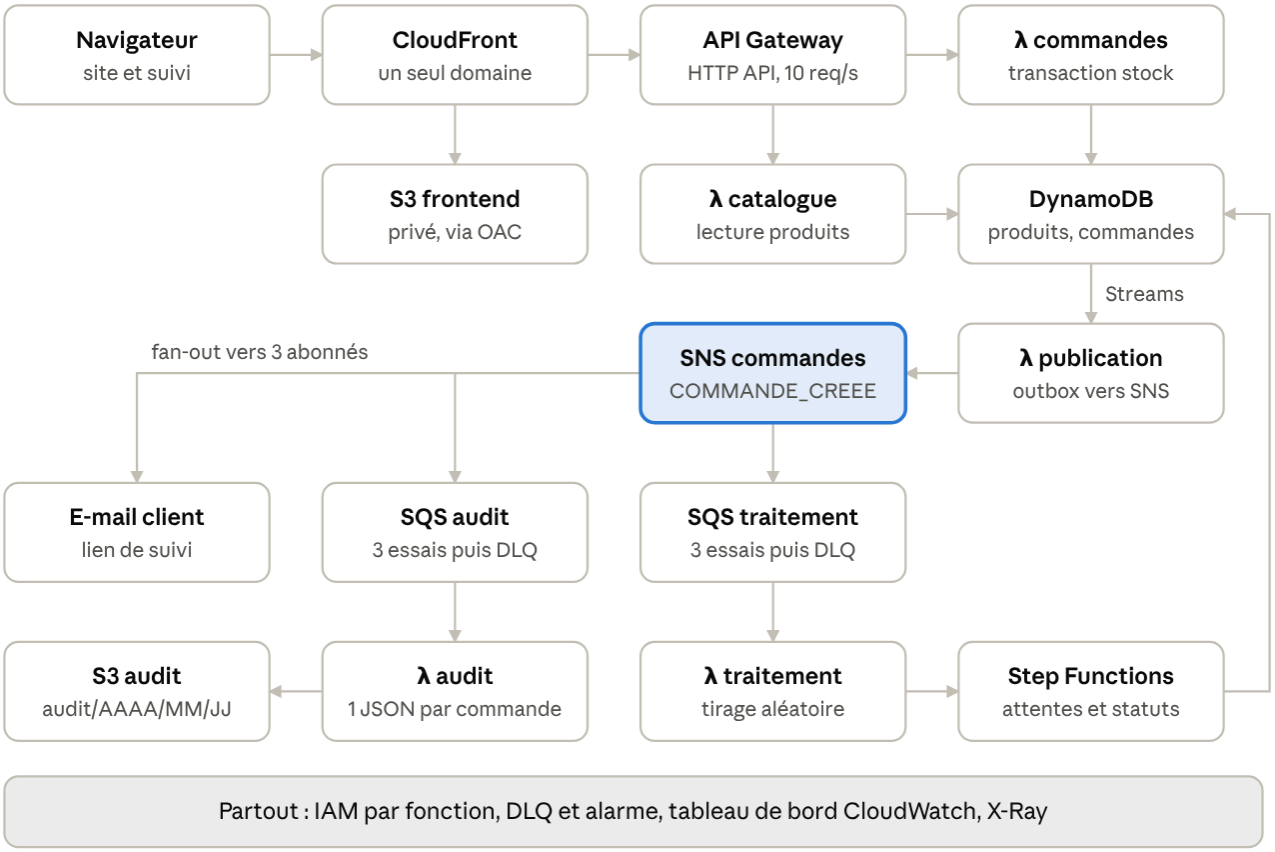
Ce document décrit en détail la **solution 1 (serverless)** de la boutique SHOP'IMT : ce qui est déployé, comment chaque pièce fonctionne, pourquoi ces choix ont été faits, et dans quel ordre le travail a été mené.

Le sujet (projet AWS FISA, R. Boulkroune) demande une boutique en ligne avec un catalogue, des commandes, une confirmation en temps réel, un audit des commandes et un traitement interne simulé. Il exige aussi deux solutions (VM et serverless), un modèle événementiel SNS, du streaming si possible, Terraform et un coût nul.

- **Compte et région** : 204924594218, eu-west-3 (Paris).
- **Site** : https://d1rpo2pqzpwl0x.cloudfront.net
- **Code** : shop-imt-serverless.zip (43 Ko), dans le dossier aws du projet.
- **Hors périmètre** : la solution 2 (EC2, ALB, Auto Scaling), qui aura son propre document. Elle reprendra les mêmes statuts et le même événement de commande pour que les deux soient comparables.

Vue d'ensemble de l'architecture

Une commande écrite déclenche tout le reste par événements



architecture serverless · 16 composants

La ligne du haut est synchrone : le client attend la réponse. Tout ce qui est sous la table DynamoDB est asynchrone et part du flux des nouvelles commandes, avec SNS au centre.

Service	Rôle dans SHOP'IMT
CloudFront	Entrée HTTPS unique (site et API), cache en edge, en-têtes de sécurité
S3 frontend	`index.html` et `app.js`, privé, lu seulement par CloudFront
API Gateway (HTTP API)	3 routes, limitation de débit, logs d'accès
Lambda (5 fonctions)	catalogue, commandes, publication, traitement, audit
DynamoDB	Tables produits et commandes, transactions

Service	Rôle dans SHOP'IMT
DynamoDB Streams	Flux des nouvelles commandes (le streaming du sujet)
SNS	Topic commandes : 2 files et l'e-mail du client
SQS	Files traitement, audit et DLQ
Step Functions	Préparation, expédition et livraison simulées
S3 audit	Une trace JSON par commande, rangée par date
CloudWatch, X-Ray	Logs, alarme, tableau de bord, traces
IAM, Budgets	Rôles par fonction, alerte de dépense

Parcours d'une commande

Une commande traverse 10 étapes. Seules les deux premières sont synchrones (le client attend la réponse) ; tout le reste est asynchrone et piloté par des événements.

1. **Chargement du site.** Le navigateur ouvre l'adresse CloudFront. CloudFront sert `index.html` et `app.js` depuis le bucket S3 privé (mis en cache 60 s).
2. **Catalogue.** `app.js` appelle `GET /produits` sur le même domaine. CloudFront répond depuis son cache (5 s) ou transmet à API Gateway, qui appelle la λ catalogue (lecture de la table produits).
3. **Commande.** Le client choisit ses articles, saisit son e-mail et valide : `POST /commandes` avec l'e-mail et la liste `{id, quantite}`.
4. **Validation et transaction.** La λ commandes vérifie l'e-mail, 1 à 10 lignes et des quantités de 1 à 20. Elle relit les prix en base, car le prix envoyé par le navigateur n'est jamais utilisé. Une **transaction DynamoDB** décrémente chaque stock (seulement s'il suffit) et écrit la commande au statut `RECUE`. Si un stock manque, rien n'est écrit et l'API répond 409 ; sinon 201 avec le numéro de commande.
5. **Streaming.** L'écriture de la commande apparaît dans le flux **DynamoDB Streams**. Un filtre ne laisse passer que les insertions. La λ publication reconstruit l'événement `COMMANDE_CREEE` et le publie sur SNS.
6. **Diffusion SNS (fan-out).** Le topic livre en parallèle l'événement aux files `traitement` et `audit`, et un texte lisible à l'abonné e-mail dont l'adresse correspond. L'e-mail contient le lien de suivi `/?commande=<id>`.
7. **Traitement.** La λ traitement lit la file et vérifie que la commande est encore `RECUE`. Elle tire au sort un transporteur, un numéro de suivi, la durée de chaque étape (1 à 8 s) et, une fois sur quatre, un incident. Elle lance ensuite une exécution Step Functions qui

porte le numéro de la commande.

8. **Expédition simulée.** Step Functions attend chaque délai, puis met à jour le statut dans DynamoDB : `EN_PREPARATION`, `EMBALLEE`, `EXPEDIEE`, parfois `INCIDENT_TRANSPORT`, `EN_LIVRAISON`, `LIVREE`. Chaque passage ajoute une ligne datée à l'historique.
9. **Audit.** En parallèle, la λ audit écrit l'événement enrichi dans S3, sous `audit/AAAA/MM/JJ/<id>.json`.
10. **Suivi.** La page de suivi interroge `GET /commandes/{id}` toutes les 2 s et affiche la frise, le transporteur, le numéro de suivi et les services AWS actifs, jusqu'à `LIVREE`.

Mesuré le 27/10 à 09h27 : commande reçue à 09:27:00, livrée à 09:27:32 après un incident « Colis retardé au centre de tri », transporteur Chronopost.

Conception détaillée par composant

Chaque composant fait une seule chose. Ils ne se parlent qu'à travers des événements ou des tables, ce qui permet d'en changer un sans toucher aux autres.

CloudFront

- Deux origines : le bucket S3 (Origin Access Control, signé en SigV4) et API Gateway (HTTPS seulement, TLS 1.2).
- `/` vers S3, avec la politique de cache `CachingOptimized`. `/produits` vers l'API avec un cache maison de 5 s. `/commandes*` vers l'API sans cache, en transmettant le corps, la query et les en-têtes (`AllViewerExceptHostHeader`).
- Politique `SecurityHeadersPolicy` (HSTS, anti-iframe, nosniff), HTTP/2 et HTTP/3, `PriceClass_100`.
- Pourquoi : un seul domaine pour le site et l'API, donc pas de CORS et pas d'URL d'API à injecter dans le front.

API Gateway (HTTP API)

L'étape `$default` se déploie automatiquement. Elle est limitée à 10 requêtes/s (rafale de 20) et ses accès sont journalisés en JSON dans CloudWatch.

Route	Lambda	Réponses
`GET /produits`	catalogue	200 : liste `{id, nom, prix, stock}`
`POST /commandes`	commandes	201 : `{order_id, statut, total, articles}` ; 400 : validation ou produit inconnu ; 409 : stock insuffisant

Route	Lambda	Réponses
`GET /commandes/{id}`	commandes	200 : statut, historique, transporteur, numéro de suivi ; 400 : identifiant invalide ; 404 : inconnue

Fonctions Lambda

Toutes tournent en Python 3.12 sur arm64 (Graviton), avec 128 Mo de mémoire et X-Ray actif. Leurs logs JSON sont gardés 7 jours, et elles gèrent les échecs message par message (`ReportBatchItemFailures`).

Fonction	Déclencheur	Droits IAM	Points clés
catalogue	API	`Scan` sur produits	Lecture paginée, tri par id
commandes	API	`BatchGetItem` , `UpdateItem` sur produits ; `PutItem` , `GetItem` sur commandes	Transaction, prix côté serveur, aucun appel SNS
publication	DynamoDB Streams (insertions)	lecture du flux, `sns:Publish` , envoi vers la DLQ	Message double (JSON pour les files, texte pour l'e-mail), s'arrête au premier échec pour garder l'ordre
traitement	SQS traitement, lots de 5	`GetItem` , `UpdateItem` sur commandes ; `states:StartExecution`	Tirage au sort, idempotence par nom d'exécution
audit	SQS audit, lots de 10	`s3:PutObject` sur `audit/*`	Clé rangée par date

Modèle de données DynamoDB

La table **produits** a pour clé `id` et contient `nom` , `prix` et `stock` . Ses 6 produits initiaux sont créés par Terraform, qui ne remet pas le stock à zéro à chaque apply (`ignore_changes`).

La table **commandes** a pour clé `order_id` , avec un flux en `NEW_IMAGE` :

Attribut	Contenu	Écrit par
order_id	UUID v4	commandes

Attribut	Contenu	Écrit par
client_email	adresse en minuscules	commandes
articles	JSON des lignes, prix unitaire figé	commandes
total	nombre	commandes
statut	`RECUE` puis les étapes suivantes	commandes, puis Step Functions
cree_le	date ISO 8601 UTC	commandes
historique	liste de `<date> <STATUT> | <détail>`	commandes, puis Step Functions
transporteur, numero_suivi	texte	traitement

Événement COMMANDE_CREEE

```
{"type": "COMMANDE_CREEE", "order_id": "…", "client_email": "…",
 "articles": [{"id": "p001", "nom": "T-shirt IMT", "quantite": 2,
"prix_unitaire": "19.90"}],
 "total": "49.30", "cree_le": "2026-10-27T09:27:00+00:00"}
```

Le message SNS porte aussi les attributs `client_email` (qui sert de filtre à l'abonnement e-mail) et `type`.

SNS et SQS

- Topic `shopimt-sls-commandes`. Les files le reçoivent en livraison brute (*raw delivery*). L'abonnement e-mail filtre sur `client_email` et doit être confirmé par son destinataire.
- Les files sont chiffrées (SSE-SQS). `traitement` et `audit` ont un délai de visibilité de 60 s et basculent en `dlq` après 3 réceptions. La DLQ garde les messages 14 jours.
- Seul le topic peut écrire dans les files (condition `aws:SourceArn`).

Step Functions « expedition »

- L'entrée est `{order_id, etapes: [{statut, delai, detail}]}`.
- Un état `Map` joue les étapes une par une. Chaque étape attend son délai (`Wait`), puis appelle directement `dynamodb:updateItem`, sans Lambda, avec 3 essais en cas d'erreur.

- Une commande consomme environ 13 transitions d'état, sur les 4 000 gratuites par mois.
- Pourquoi : attendre ne coûte rien, contrairement à une Lambda qui dort, et l'exécution se lit comme un graphe dans la console, ce qui est idéal pour la démo.

Stockage S3

- `shopimt-sls-frontend-<compte>` : privé, lisible seulement par la distribution CloudFront.
- `shopimt-sls-audit-<compte>` : privé, les traces `audit/` expirent après 90 jours.
- Les deux sont en `force_destroy`, pour qu'un `terraform destroy` les vide et les supprime.

Supervision

- L'alarme `shopimt-sls-dlq-non-vide` se déclenche dès qu'un message arrive en DLQ.
- Le tableau de bord `shopimt-sls-tableau-de-bord` suit les requêtes, erreurs et latences p50/p95 de l'API, les invocations et erreurs Lambda, les files en attente, les publications SNS et la capacité DynamoDB consommée.
- X-Ray montre la carte des services et le temps passé dans chaque appel AWS.

Sécurité, résilience, coût

Sécurité

- **Moindre privilège** : un rôle IAM par Lambda et un pour Step Functions, limités aux actions et aux ressources nécessaires.
- **Rien de public** : les buckets sont privés, et le site n'est lisible qu'à travers CloudFront.
- **Chiffrement** : HTTPS de bout en bout. Les données sont chiffrées au repos dans DynamoDB, S3 et SQS, avec des clés gérées par AWS (gratuites).
- **Application** : validation stricte des entrées et prix recalculés côté serveur. Le front n'injecte jamais de HTML, ce qui évite les failles XSS. L'API est limitée en débit, et CloudFront ajoute des en-têtes de sécurité.
- **Déploiement** : le déploiement passe par l'utilisateur IAM dédié `shopimt-deployer`, jamais par le compte root. Sa clé d'accès est créée pour un déploiement puis supprimée aussitôt.

Résilience

- **Haute disponibilité** : tous les services sont régionaux et répartis sur plusieurs zones de disponibilité par AWS. CloudFront sert le site depuis ses points de présence.
- **Cohérence** : la transaction DynamoDB empêche la survente et les commandes à moitié écrites.
- **Pas d'événement perdu** : avec le pattern *outbox*, une commande écrite est forcément publiée, car le flux réessaie jusqu'à 3 fois avant la DLQ.
- **Erreurs isolées** : un message en échec ne bloque pas les autres. Après 3 échecs, il part en DLQ et déclenche l'alarme.
- **Idempotence** : une relivraison ne relance rien. L'exécution Step Functions porte le nom de la commande, le transporteur n'est fixé qu'une fois, et la clé d'audit est l'identifiant de la commande.

Coût

Service	Réglage	Offre gratuite utilisée
Lambda	128 Mo, arm64, aucune attente active	1 M d'appels et 400 000 Go-s par mois
DynamoDB	provisionné 5/5 par table, flux	25 RCU / 25 WCU / 25 Go ; lecture du flux par Lambda gratuite
Step Functions	Standard, états Wait	4 000 transitions par mois
SNS, SQS	standard	1 M de requêtes par mois chacun, 1 000 e-mails
CloudFront	PriceClass_100	1 To et 10 M de requêtes par mois
API Gateway	HTTP API	offre gratuite ou crédits du compte
CloudWatch	logs 7 j, 1 alarme, 1 tableau de bord	5 Go de logs, 10 alarmes, 3 tableaux de bord
X-Ray	Lambdas et Step Functions	100 000 traces par mois

Ne sont pas déployés, car payants : MSK, Kinesis, WAF, clés KMS client, Athena, Route 53 et NAT Gateway. CloudTrail est prêt mais désactivé par défaut (`activer_cloudtrail`), car son stockage S3 coûterait quelques centimes par mois.

Correspondance avec le sujet

Toutes les exigences du sujet sont couvertes pour la partie serverless. Le streaming, demandé « si possible », passe par DynamoDB Streams ; la solution EC2 reste à faire.

Exigence du sujet	Réponse dans SHOP'IMT
Parcourir un catalogue via une interface Web	Site S3 + CloudFront, `GET /produits`
Commander un ou plusieurs produits	`POST /commandes`, transaction DynamoDB sur le stock
Recevoir une confirmation en temps réel	Réponse immédiate, e-mail SNS avec lien de suivi, page de suivi en direct
Enregistrer les commandes pour audit et reporting	SQS audit vers S3, rangé par date, prêt pour Athena
Déclencher emballage et expédition simulés	SQS, Lambda puis Step Functions, avec aléatoire et incidents
Services serverless	Lambda, API Gateway, DynamoDB, SNS, SQS, Step Functions, S3, CloudFront
Modèle événementiel SNS	Un topic, 3 abonnés, filtre sur l'e-mail
Streaming (Kafka, MSK) si possible	DynamoDB Streams ; MSK présenté en architecture cible car payant
Terraform	100 % de l'infrastructure, données initiales comprises
Free Tier, coût zéro	Réglages détaillés dans la partie coût
Haute disponibilité	Services managés multi-AZ, CloudFront
README pour lancer et tester	README, tests unitaires et `scenario.sh`
Solution sur machines virtuelles (EC2)	Solution 2, à venir

Journal de réalisation

Le travail s'est fait en 8 étapes, du 26 au 27 octobre. Chacune a été vérifiée avant de passer à la suivante.

Date	Étape	Ce qui a été fait, et pourquoi
27/10, 09h28	Redéploiement des améliorations	10 ressources ajoutées, 14 modifiées, 1 supprimée. Commande de test livrée en 32 s avec un incident ; DynamoDB, audit S3 et Step Functions vérifiés, DLQ vide. Clé de déploiement supprimée, nouvel état et nouveau zip (43 Ko) sauvegardés.
27/10, 08h53	Améliorations demandées par Louison	Streaming DynamoDB Streams et λ publication (outbox). API servie par CloudFront, cache du catalogue, en-têtes de sécurité. Expédition aléatoire avec Step Functions et statut `LIVREE`. Lien de suivi dans l'e-mail et page de suivi. Refonte du site, testée avec Playwright sur une fausse API (clair, sombre, mobile). Tableau de bord CloudWatch, X-Ray, CloudTrail en option. Tests unitaires étendus.
27/10, 08h46	Premier déploiement	La clé AWS du poste de Louison était invalide, donc le déploiement s'est fait depuis l'environnement de Claude avec l'utilisateur dédié `shopimt-deployer`. 57 ressources créées. Scénario réussi : erreurs 400 et 409, commande expédiée, audit S3.
27/10, 08h33	Vérification du livrable	Tests unitaires OK. `terraform validate` OK avec le provider AWS 5.80 et 6.70. Corrections : formatage, provider borné à `< 7.0`, budget désactivé par défaut puisque le compte en avait déjà deux.
27/10, matin	Renommage en SHOP'IMT	Fait par Louison dans tout le livrable.
27/10, matin	Première version serverless	4 Lambdas, API Gateway, DynamoDB, SNS, 2 files SQS, S3, CloudFront, DLQ, alarme, budget. Tests unitaires avec un faux SDK AWS.
27/10, 07h58	Connexion du compte AWS	Connecteur AWS branché, ressources de test `boutique-test-*`, budget à 1 $, comptes IAM pour Gabin et Félix.

Date	Étape	Ce qui a été fait, et pourquoi
26/10	Pistes d'architecture	3 pistes comparées. Recommandation : un cœur événementiel SNS → SQS commun aux deux solutions, avec Kafka sur une petite instance EC2 plutôt que MSK pour la solution 2.

Mise à jour du 27/10, 09h44 : identité visuelle. Le site reprend les couleurs de la charte IMT (bleu IMT #00B8DE, bleu marine #14223C, gris #EDF3F4) et la typographie Helvetica. Il a un logo SHOP'IMT original et un visuel SVG détaillé pour chacun des 6 articles (36 Ko au total), publiés par Terraform dans `img/`. Le logo officiel IMT n'est pas utilisé, car la charte interdit de le modifier ou de l'associer à une autre marque.

Déployer, tester, démontrer, supprimer

Déployer

```
cd shop-imt/serverless
cp terraform.tfvars.example terraform.tfvars   # notification_email en
minuscules
terraform init
terraform apply                                # 5 à 10 min la première fois
```

Ensuite, confirmer l'abonnement SNS dans l'e-mail « AWS Notification - Subscription Confirmation ».

L'infrastructure actuelle a été déployée depuis l'environnement de Claude. Son état Terraform est dans le dossier `aws/shop-imt-serverless-etat` du projet. Pour la modifier ou la supprimer depuis un poste, il faut copier `terraform.tfstate` dans `serverless/` avant toute commande, sinon Terraform voudra tout recréer en double.

Tester

- `python3 tests/test_lambdas.py` : tests sans AWS (validation, stock insuffisant et annulation, publication par le flux, tirage aléatoire, idempotence, audit).
- `./tests/scenario.sh ton.email` : scénario de bout en bout. Il passe par le catalogue, deux erreurs (400, 409) et une commande suivie jusqu'à `LIVREE`, puis vérifie DynamoDB, l'audit S3 et l'exécution Step Functions.

Démontrer en 5 minutes

1. Commander deux articles sur le site : la page de suivi s'ouvre et avance toute seule.

2. Ouvrir l'e-mail de confirmation et cliquer sur le lien de suivi.
3. Dans la console : le graphe de l'exécution Step Functions, le fichier d'audit dans S3, le tableau de bord CloudWatch et la carte X-Ray.
4. Demander plus de gourdes qu'il n'y en a en stock : refus immédiat, et le stock ne bouge pas.

Supprimer

`terraform destroy`. Les buckets sont vidés automatiquement.

Limites et pistes

- **Rejouer l'historique des événements** : DynamoDB Streams garde 24 h ; MSK Serverless ou Kinesis garderaient tout (payants).
- **Vrais e-mails clients** : SES, car SNS impose que chaque adresse confirme son abonnement.
- **Comptes clients** : Cognito et une autorisation JWT, pour que seul le client voie sa commande. Aujourd'hui, c'est le numéro de commande aléatoire qui sert de secret.
- **Reporting** : Athena puis QuickSight sur le bucket d'audit.
- **Protection** : WAF devant CloudFront (payant).

Mise à jour : e-mail HTML (SES), comptes clients (Cognito) et version mobile

E-mail de confirmation avec Amazon SES

SNS ne sait envoyer que du texte brut. L'abonnement e-mail SNS a donc été remplacé par un abonné Lambda, `notification`, qui construit un e-mail HTML et l'envoie avec **Amazon SES v2** (`ses:SendEmail`).

- Contenu : logo SHOP'IMT, photo de chaque article (PNG servis par CloudFront, car les messageries n'affichent pas le SVG), quantités, total, bouton « Suivre ma commande → » vers `/?commande=<id>`, et une version texte de secours.
- Sécurité : tous les champs sont échappés (`html.escape`), testé contre l'injection de HTML.
- Terraform : `aws_sesv2_email_identity` pour l'expéditeur et pour chaque adresse de `emails_verifies`.
- Sandbox SES : seules les adresses vérifiées reçoivent l'e-mail. Un destinataire non vérifié est journalisé (`EMAIL_REFUSE`) puis ignoré, sans relance. Vérifié le 27/10 : l'e-mail de la commande `c67ccf7c` a été envoyé (`EMAIL_ENVOYE`).

Comptes clients avec Amazon Cognito

1. **User pool** `shopimt-sls-clients` : inscription par e-mail, code de confirmation envoyé par Cognito, mot de passe de 8 caractères minimum.
2. **Client public** `shopimt-sls-site` (sans secret) avec les flux `USER_PASSWORD_AUTH` et `REFRESH_TOKEN_AUTH`. Le site appelle directement l'API Cognito (`SignUp` , `ConfirmSignUp` , `InitiateAuth`).
3. **Authorizer JWT** sur l'API HTTP : les routes `POST /compte/commandes` et `GET /compte/commandes` refusent toute requête sans jeton valide (401), sans écrire une ligne de code de vérification.
4. **DynamoDB** : index secondaire global `par-client` (`client_sub` + `cree_le`) sur la table des commandes. « Mes commandes » est une simple requête sur cet index, la plus récente en premier.
5. **CloudFront** : comportement `/compte/*` sans cache, avec l'en-tête `Authorization` transmis à l'API.
6. **config.json** publié sur S3 par Terraform (région et identifiant du client Cognito) : le site n'a aucune valeur codée en dur.

Une commande passée connecté prend l'e-mail du jeton (il ne peut pas être falsifié) et enregistre `client_sub` . Test réel du 27/10 : 401 sans jeton, puis commande créée et retrouvée dans l'historique avec le jeton.

Version mobile (iPhone)

Sur téléphone, la navigation passe dans une barre d'onglets en bas (Boutique, Suivre, Mon compte), avec un panier flottant qui affiche le nombre d'articles et le total. Le site respecte les zones de l'encoche et de la barre d'accueil (`viewport-fit=cover` , `safe-area-inset`), agrandit les boutons à 44 px, et peut être ajouté à l'écran d'accueil (icône et `manifest.webmanifest`). Ce sont des changements HTML/CSS : ils n'ajoutent aucun service AWS.

Services AWS ajoutés et coût

Service	Rôle	Gratuité
Amazon SES	E-mail HTML de confirmation	3 000 messages / mois pendant 12 mois
Amazon Cognito	Comptes clients, jetons JWT	10 000 utilisateurs actifs / mois

Service	Rôle	Gratuité
API Gateway (authorizer JWT)	Protection des routes du compte	inclus dans l'API HTTP
DynamoDB (index secondaire)	Historique par client	dans les 25 RCU / WCU gratuits